Sécurité des systèmes

Système de gestion des mises à jour en PowerShell

Projet Patch Management

Arshdeep SINGH
Fahed TADRIST

Enseignant : Salim Benayoune
EFREI Paris
Rapport remis le 31 octobre 2026

Dépôt GitHub privé : github.com/Arshdeep931/patch-management-powershell

Table des matières

1. Introduction 3

2. Architecture de la solution 3

2.1 Environnement de test 3
2.2 Organisation du dépôt 4
2.3 Fonctionnement général 5

3. Fonctionnalités réalisées 6

Partie 1 : disponibilité des postes 6
Partie 2 : inventaire du parc 6
Partie 3 : inventaire des correctifs 6
Partie 4 : contrôle de conformité 6
Partie 6 : rapport de sécurité 6
Partie 7 : notification 6

4. Choix techniques 7

5. Tests effectués et résultats 7

5.1 Parties 1 à 4 8
5.2 Partie 6 : rapport de sécurité 9
5.3 Partie 7 : notification 12

6. Sécurisation de la solution 14

6.1 Risques de WinRM 14
6.2 Qui peut ouvrir une session distante 15
6.3 Limiter les privilèges du compte 15
6.4 Accès aux fichiers produits 16
6.5 Règles du pare-feu 16
6.6 Authentification et Kerberos 17
6.7 Just Enough Administration (JEA) 17
6.8 Conservation des rapports et des journaux 18

7. Difficultés rencontrées 18

8. Limites et améliorations possibles 18

9. Conclusion 19

Annexes : réponses aux questions du sujet 20

Annexe A : Partie 1, préparation de l'environnement 20
Annexe B : Partie 4, contrôle de conformité 21
Annexe C : Partie 7, notification 21
Annexe D : Partie 8, sécurisation de la solution 22

1. Introduction

Ce projet consiste à contrôler à distance l'état des mises à jour Windows d'un parc de postes, depuis un poste d'administration, avec des scripts PowerShell. L'idée est celle d'un petit système de Patch Management : on interroge chaque poste, on vérifie qu'il possède bien les correctifs de sécurité exigés, on produit un rapport, et on prévient l'administrateur seulement quand une machine pose problème.

Nous avons travaillé à deux (Arshdeep Singh et Fahed Tadrist) sur un environnement de test composé de deux machines virtuelles. Ce rapport suit les parties du sujet, dont la numérotation est reprise telle quelle (le sujet ne comporte pas de partie 5) : disponibilité des postes, inventaire, correctifs, conformité, rapport de sécurité, notification, puis sécurisation de la solution. Viennent ensuite les choix techniques, les tests, les difficultés rencontrées et les limites de notre travail.

L'ensemble du code, des fichiers de configuration et de notre journal de bord se trouve dans le dépôt GitHub privé du projet.

2. Architecture de la solution

2.1 Environnement de test

Nous avons monté deux machines virtuelles Windows 11 Professionnel (ARM64) avec VMware Fusion, sur un Mac à puce Apple Silicon. Les deux machines sont sur le réseau 192.168.10.0/24, ne sont pas dans un domaine (groupe de travail) et n'ont pas d'accès à Internet.

Machine	Adresse IP	Rôle	Remarque
ADMIN01	192.168.10.10	Poste d'administration	Lance tous les scripts
PC01	192.168.10.11	Poste administré	Seul poste réel du parc
PC02	192.168.10.12	Poste déclaré	Listé dans computers.txt, VM inexistante
PC03	192.168.10.13	Poste déclaré	Listé dans computers.txt, VM inexistante

PC02 et PC03 n'existent pas vraiment : ils sont seulement écrits dans le fichier computers.txt. Nous les avons gardés volontairement pour tester le cas d'un poste injoignable, que le sujet demande de gérer. La communication passe par WinRM (PowerShell Remoting) en HTTP sur le port 5985. Comme il n'y a pas de domaine, l'adresse de PC01 est déclarée dans TrustedHosts sur ADMIN01.

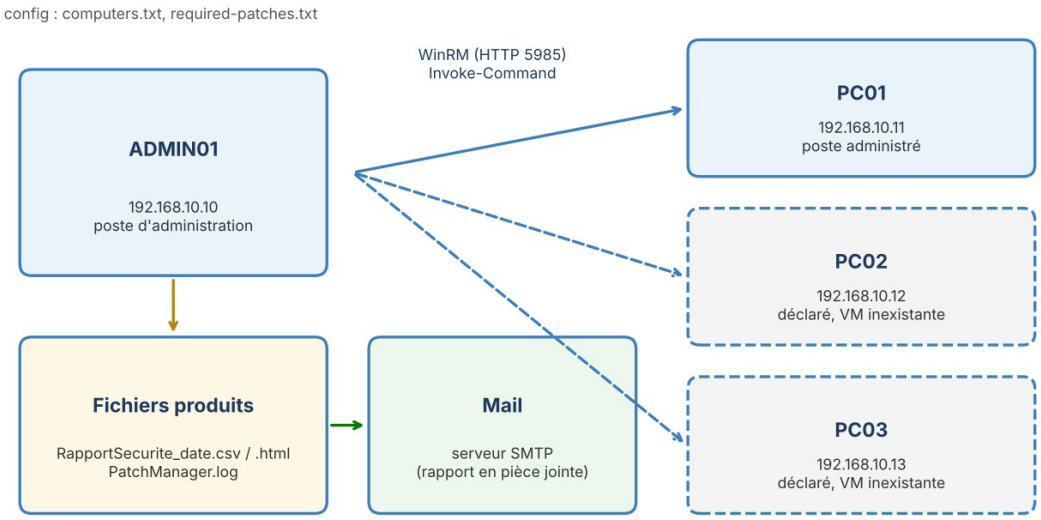


Figure 1 : Architecture de la solution

2.2 Organisation du dépôt

Le dépôt est rangé en dossiers, avec un commit par partie du sujet :

```
scripts/  Test-ParcDisponibilite.ps1, Get-InventaireParc.ps1,
          Get-InventairePatches.ps1, Test-Conformite.ps1,
          PatchManager.ps1, Send-Notification.ps1, Start-SmtpTest.ps1
config/   computers.txt, required-patches.txt
docs/     journal-de-bord.md, environnement.md, screenshots/
rapport/  rapport au format PDF et Word
reports/  rapports CSV/HTML et journal (non versionnés)
```

Le fichier computers.txt contient une ligne par machine au format Nom;Adresse IP. Le fichier required-patches.txt liste les correctifs obligatoires, un par ligne. Le dossier reports et le fichier d'identifiants de messagerie (config/smtp-cred.xml) sont exclus de Git avec le .gitignore, pour ne jamais publier de rapports ni de secret.

Dans le dépôt, les scripts calculent leurs chemins à partir de leur propre emplacement ($PSScriptRoot). Sur nos machines virtuelles, nous avons copié les scripts à plat sur le Bureau avec les fichiers de configuration à côté : la logique est la même, seuls les chemins changent.

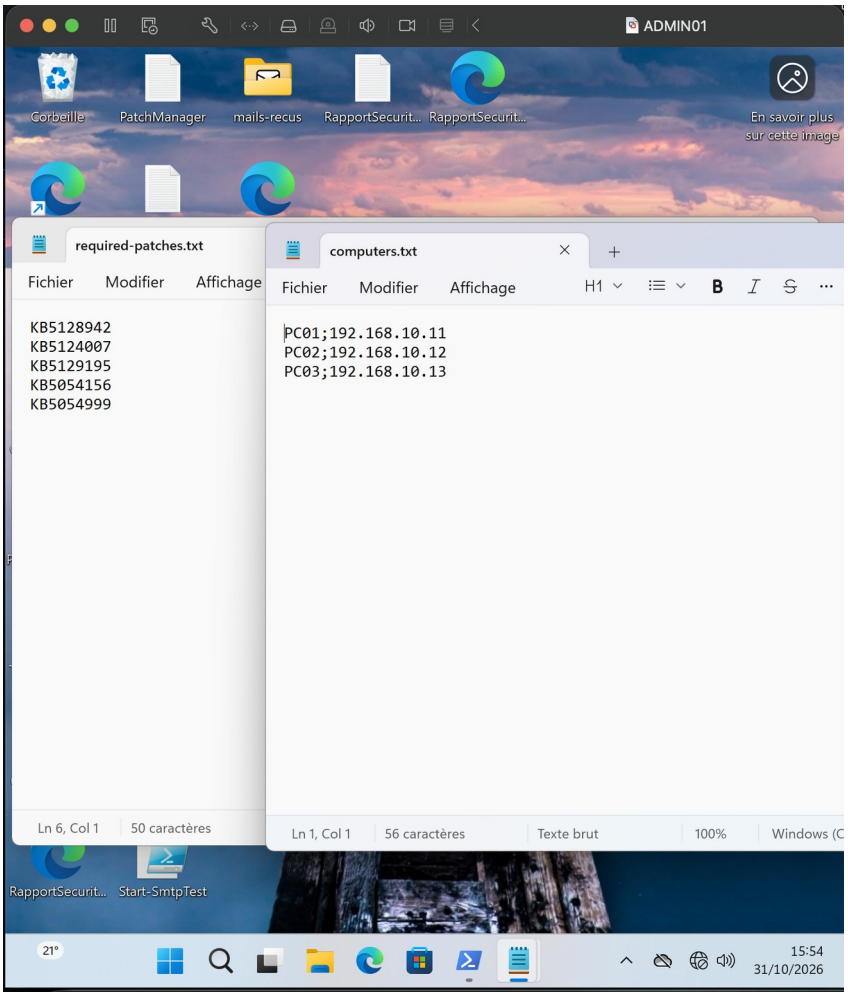


Figure 2 : Fichiers de configuration sur ADMIN01 : required-patches.txt (avec la KB de test KB5054999) et computers.txt

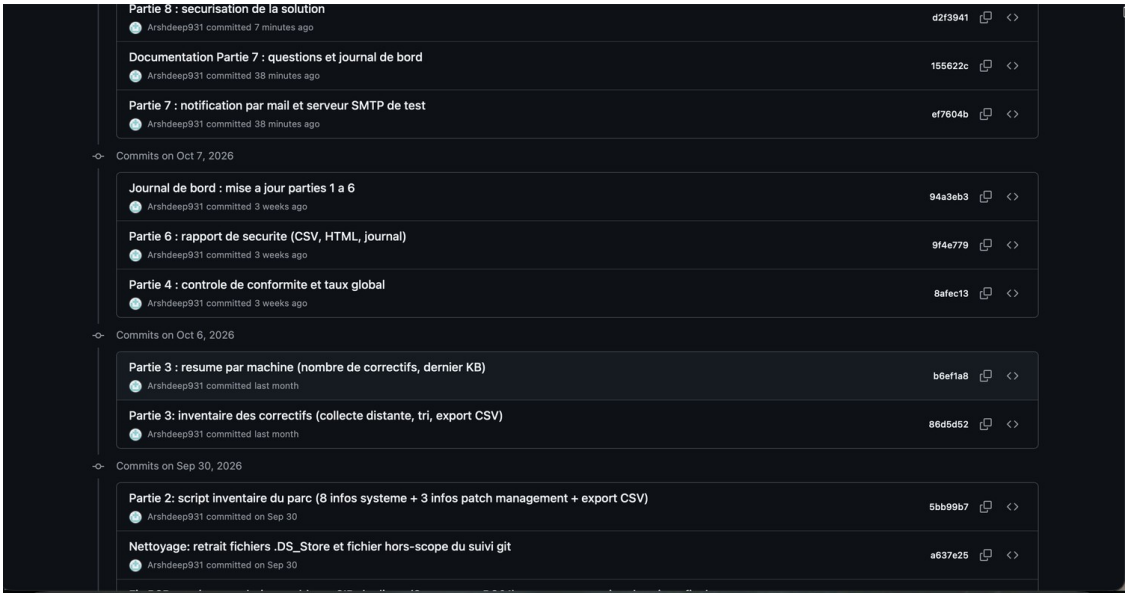


Figure 3 : Historique des commits du dépôt GitHub

2.3 Fonctionnement général

PatchManager.ps1 lit computers.txt et required-patches.txt, interroge chaque poste en WinRM, calcule la conformité et écrit un rapport CSV, un rapport HTML et une ligne de journal par étape.

Send-Notification.ps1 relit ensuite le dernier rapport CSV et, s'il y a au moins un poste NON CONFORME ou INACCESSIBLE, envoie un mail à l'administrateur avec le rapport en pièce jointe.

3. Fonctionnalités réalisées

Partie 1 : disponibilité des postes

Test-ParcDisponibilite.ps1 lit computers.txt et affiche pour chaque machine son nom, son adresse IP et son état (Disponible ou Indisponible). Le test se fait avec Test-Connection (un seul paquet, mode silencieux). Cette partie comprend aussi la mise en place de WinRM sur PC01 et les réponses aux questions de théorie (annexe A).

Partie 2 : inventaire du parc

Get-InventaireParc.ps1 collecte pour chaque poste accessible des informations système (modèle, version du BIOS, version de Windows, architecture, dernier démarrage, mémoire, espace disque) et des informations utiles au suivi des mises à jour (nombre de correctifs, état du service Windows Update). Un poste qui ne répond pas est noté Inaccessible. Le résultat est exporté dans InventaireParc.csv.

Partie 3 : inventaire des correctifs

Get-InventairePatches.ps1 récupère les correctifs installés avec Get-HotFix : numéro de KB, description et date d'installation. Le script affiche ensuite un résumé par machine (nombre de correctifs et dernier correctif installé, avec Group-Object) et exporte tout dans PatchesInventory.csv.

Partie 4 : contrôle de conformité

Test-Conformite.ps1 compare les correctifs installés avec la liste de required-patches.txt. Un poste est CONFORME seulement si toutes les KB requises sont présentes, NON CONFORME s'il en manque (la liste des KB manquantes est indiquée), et INACCESSIBLE s'il ne répond pas. Le taux de conformité du parc est calculé sur les postes accessibles uniquement. La réponse à la question du sujet est en annexe B.

Partie 6 : rapport de sécurité

PatchManager.ps1 regroupe toutes les étapes précédentes. Pour chaque poste, il produit un objet de 12 champs : Poste, IP, Accessibilité, Windows, DernierDémarrage, ÉtatWU, NbKbRequises, NbKbManquants, ListeKbManquants, ÉtatConformité, DateEtHeureContrôle et Erreur. Il calcule ensuite une synthèse de 7 indicateurs (total de postes, accessibles, inaccessibles, conformes, non conformes, total de KB manquantes, taux de conformité) et la liste des postes qui demandent une intervention. Les résultats sont écrits dans un rapport CSV, un rapport HTML et un fichier journal PatchManager.log, avec la date dans le nom des rapports.

Partie 7 : notification

Send-Notification.ps1 lit le dernier rapport CSV. S'il n'y a aucune anomalie, il écrit seulement une ligne dans le journal et s'arrête, sans envoyer de mail. Sinon, il construit le message demandé par le sujet (objet « [PATCH MANAGEMENT] Anomalies détectées », date du contrôle, nombre de

postes contrôlés, non conformes et inaccessibles, puis la liste des postes avec les KB manquantes) et l'envoie avec le rapport CSV en pièce jointe. Le résultat de l'envoi, réussi ou en erreur, est noté dans le journal. Aucun mot de passe n'est écrit dans le script (voir les choix techniques).

Comme les machines virtuelles n'ont pas d'accès à Internet, nous n'avons pas pu installer un serveur de test existant. Nous avons donc écrit Start-SmtpTest.ps1, un petit serveur SMTP en PowerShell qui écoute sur localhost:25, enregistre chaque mail reçu et en produit une version lisible avec la pièce jointe extraite.

4. Choix techniques

- **Un seul Invoke-Command par poste.** Toutes les données d'un poste (Windows, démarrage, Windows Update, liste des KB) sont récupérées en une connexion, puis la comparaison avec les KB requises se fait en local. Les variables locales n'existent pas dans le bloc exécuté à distance, donc c'était aussi plus simple et plus rapide.
- **Ping avant WinRM.** Un poste éteint fait attendre longtemps une connexion WinRM. Test-Connection permet de le déclarer INACCESSIBLE tout de suite, avec la raison dans la colonne Erreur.
- **Gestion des erreurs.** Les appels distants utilisent -ErrorAction Stop dans un try/catch, pour qu'un problème sur un poste n'arrête pas le contrôle des autres.
- **Postes inaccessibles hors du taux.** On ne peut pas savoir si un poste éteint est conforme. Le taux est donc conformes / (conformes + non conformes).
- **Objets PowerShell et exports.** Chaque poste est un objet, ce qui permet de trier, de filtrer, de l'afficher en tableau et de l'exporter en CSV (UTF-8) ou en HTML avec ConvertTo-Html.
- **Configuration en dehors du code.** La liste des postes et des KB obligatoires est dans des fichiers texte, modifiables sans toucher aux scripts.
- **Journal horodaté.** Une fonction Write-Log écrit une ligne « date ; message » à chaque étape, ce qui permet de retracer ce qui s'est passé.
- **Identifiants de messagerie.** S'il faut un compte pour le serveur SMTP, il est lu depuis un fichier chiffré créé avec Get-Credential et Export-Clixml (chiffrement DPAPI, lié à l'utilisateur et à la machine). Le fichier est ignoré par Git. Sans ce fichier, l'envoi se fait sans authentification, ce qui suffit pour le serveur de test local.
- **Accents dans les scripts.** Pour éviter les problèmes d'encodage entre le Mac et Windows, les caractères accentués du message et du journal sont construits avec leur code ([char]0xE9 pour é, par exemple), et les fichiers restent en ASCII.
- **Git.** Un commit par partie, avec des « git add » explicites sur les fichiers concernés, pour ne jamais ajouter par erreur un rapport ou un fichier hors sujet.

5. Tests effectués et résultats

Tous les tests ont été faits depuis ADMIN01, contre PC01 (poste réel) et PC02 et PC03 (postes déclarés mais inexistants). Pour obtenir un poste NON CONFORME, nous avons ajouté dans required-patches.txt une fausse KB, KB5054999, que PC01 ne peut pas avoir. En la retirant, PC01 devient CONFORME.

Scénario	Résultat attendu	Résultat obtenu
PC01 sans la KB de test requise	NON CONFORME, taux 0 %	Conforme à l'attendu
PC01 avec toutes les KB requises	CONFORME, taux 100 %	Conforme à l'attendu
PC02 et PC03 (inexistants)	INACCESSIBLE, hors du taux	Conforme à l'attendu
Notification avec anomalies, serveur de test lancé	Mail reçu, ligne « envoyée » dans le journal	Conforme à l'attendu
Notification, serveur de test arrêté	Pas de plantage, ligne ERREUR dans le journal	Conforme à l'attendu

5.1 Parties 1 à 4

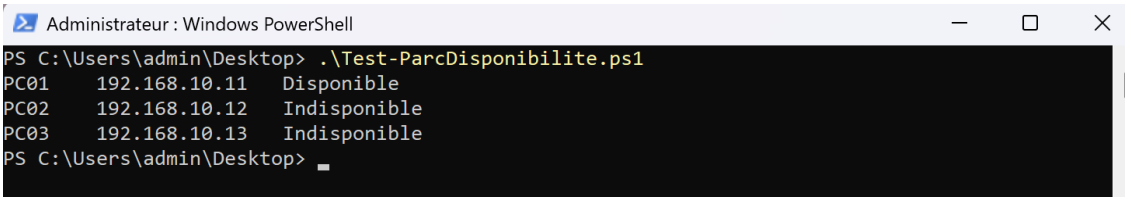


Figure 4 : Partie 1 : disponibilité des postes

PC01 répond, PC02 et PC03 sont indisponibles, comme prévu.

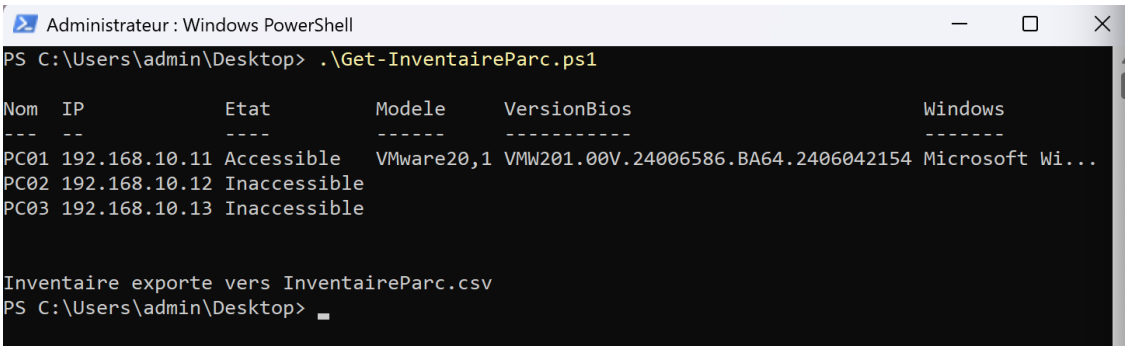


Figure 5 : Partie 2 : inventaire du parc (extrait des colonnes, le reste est dans InventaireParc.csv)

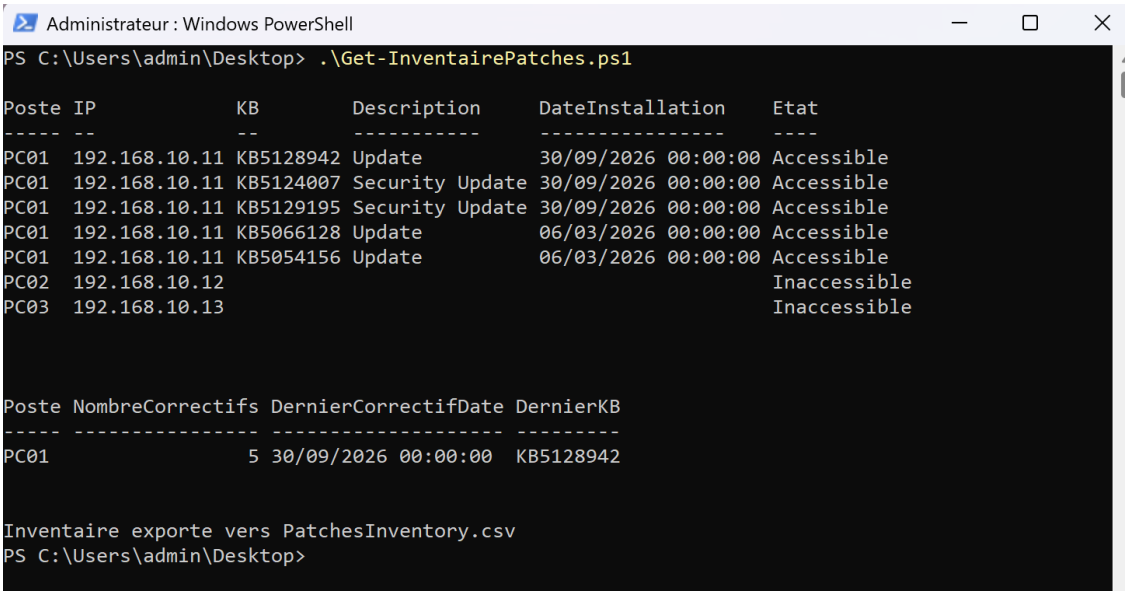


Figure 6 : Partie 3 : correctifs installés sur PC01 et résumé par machine

PC01 a 5 correctifs installés, le dernier étant KB5128942 installé le 30/09/2026.

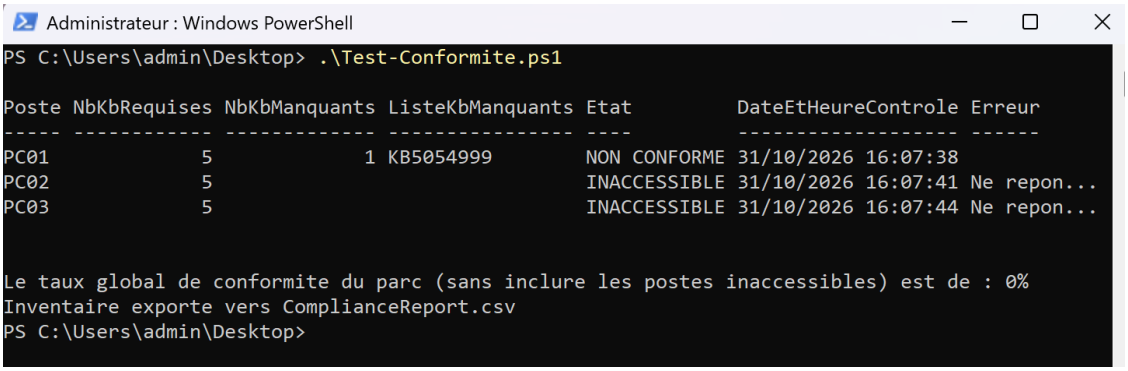


Figure 7 : Partie 4 : contrôle de conformité et taux global

5.2 Partie 6 : rapport de sécurité

Premier test, avec la KB de test dans la liste des KB requises : PC01 est NON CONFORME et le taux est de 0 %.

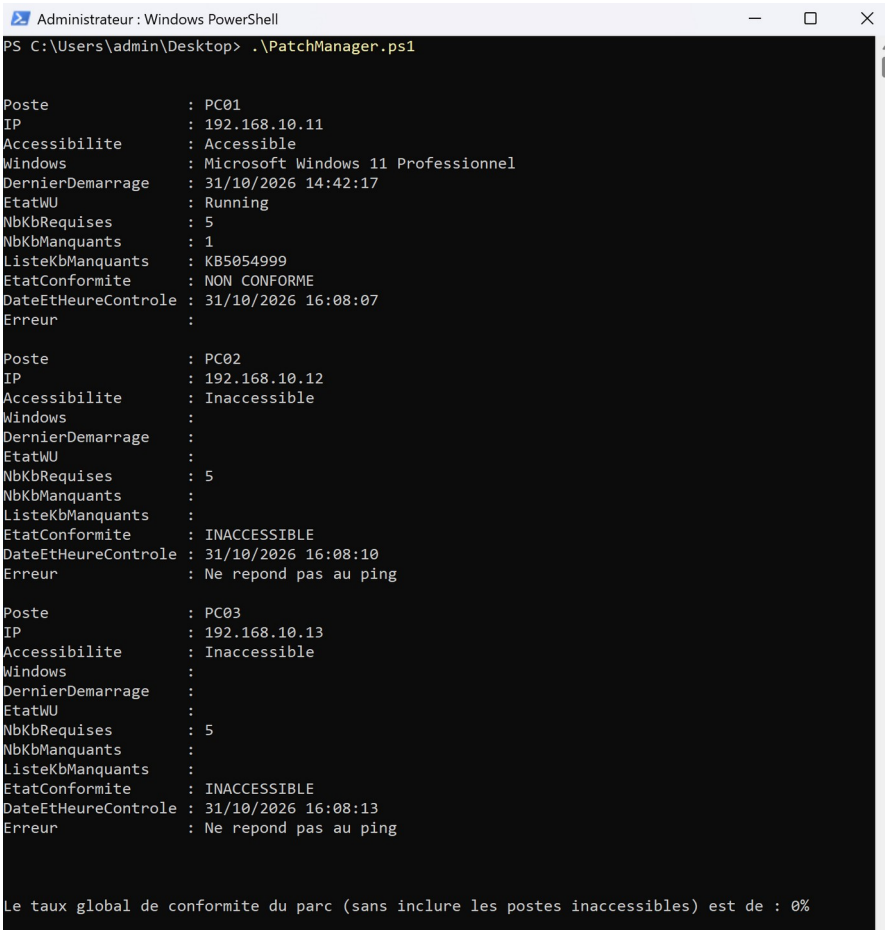


Figure 8 : PatchManager.ps1, détail par poste (cas NON CONFORME)

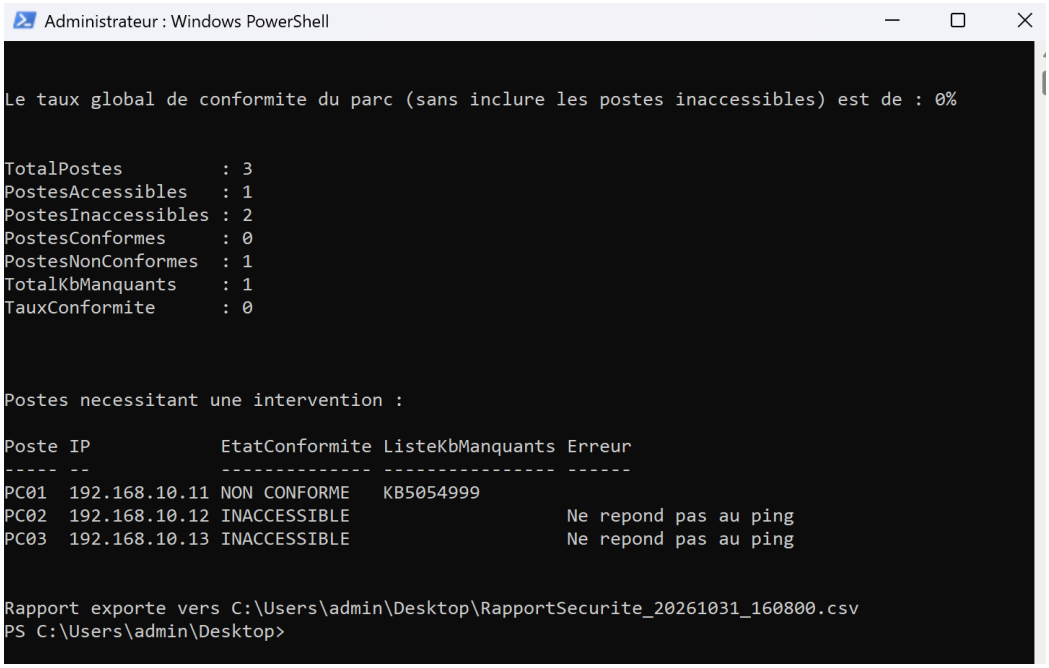


Figure 9 : PatchManager.ps1, synthèse et postes nécessitant une intervention

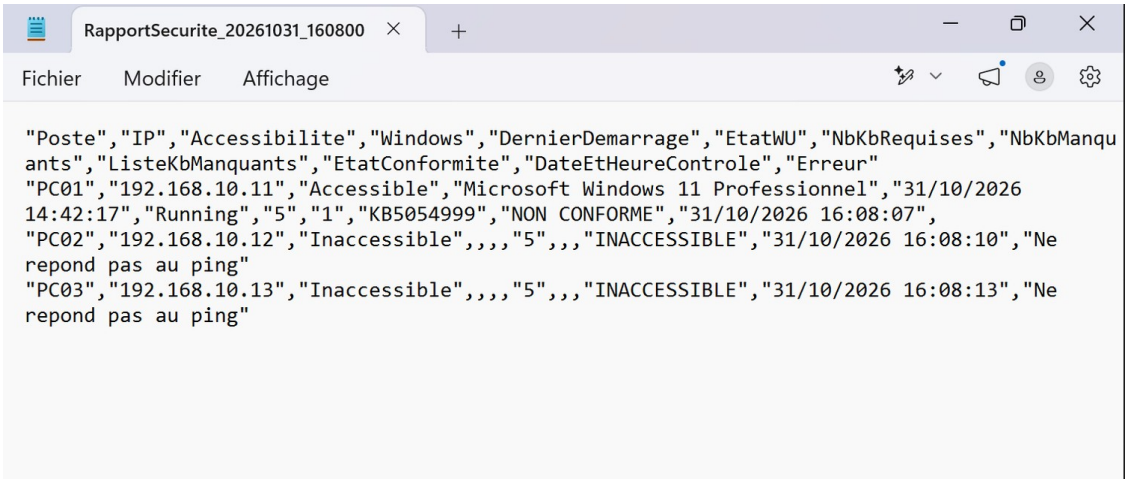


Figure 10 : Rapport CSV généré (RapportSecurite_20261031_160800.csv)

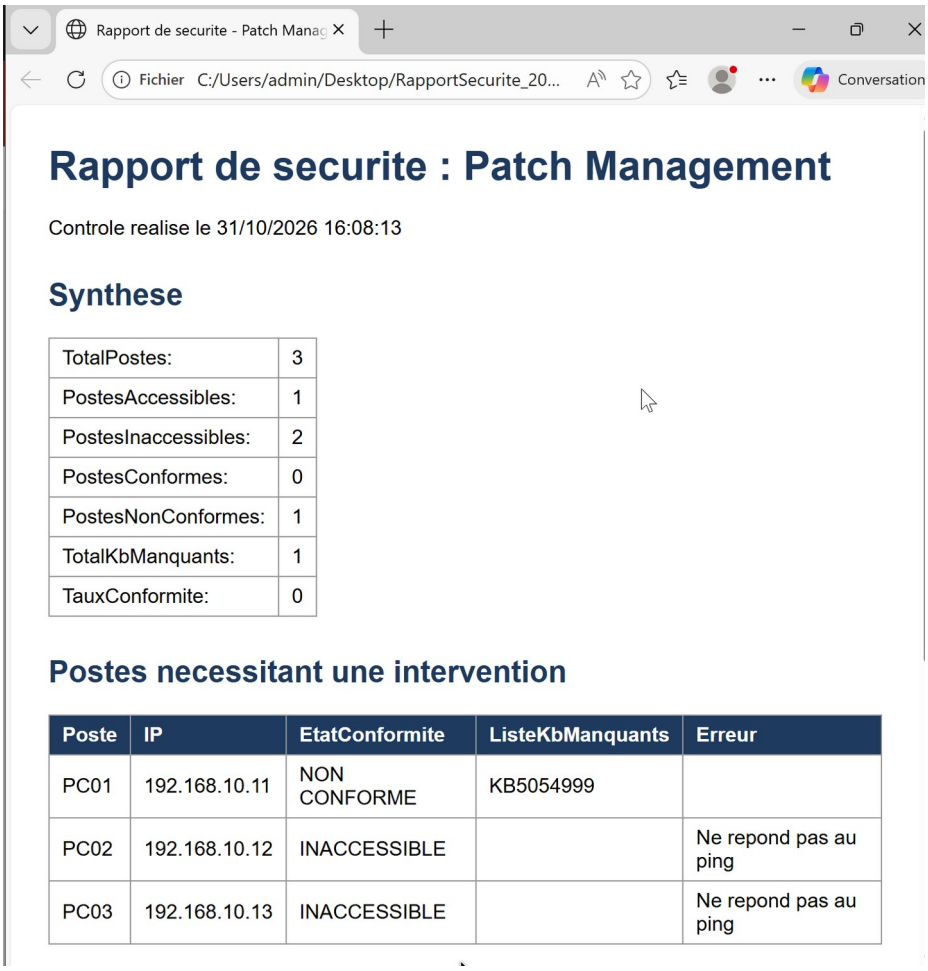


Figure 11 : Rapport HTML : synthèse et postes nécessitant une intervention



Figure 12 : Rapport HTML : détail par poste

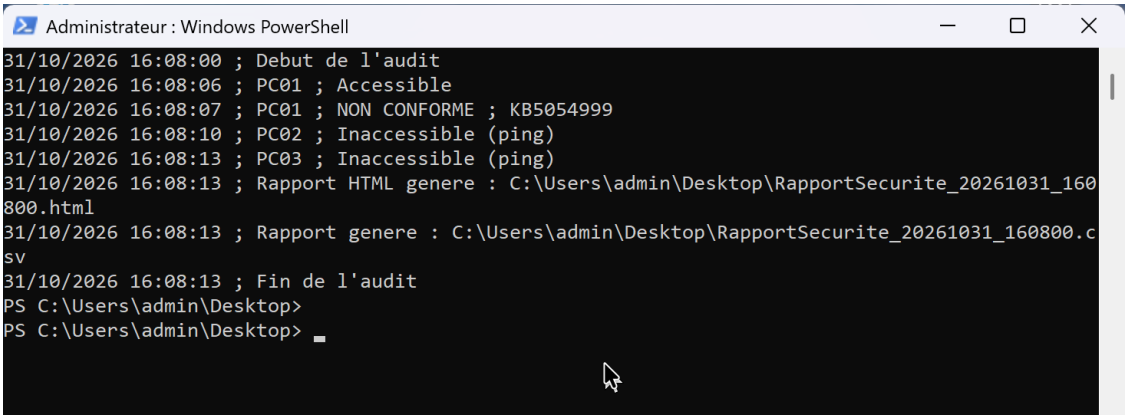


Figure 13 : Journal PatchManager.log après l'audit

Second test, sans la KB de test : PC01 est CONFORME, le taux passe à 100 % et seuls PC02 et PC03 restent dans la liste des interventions.

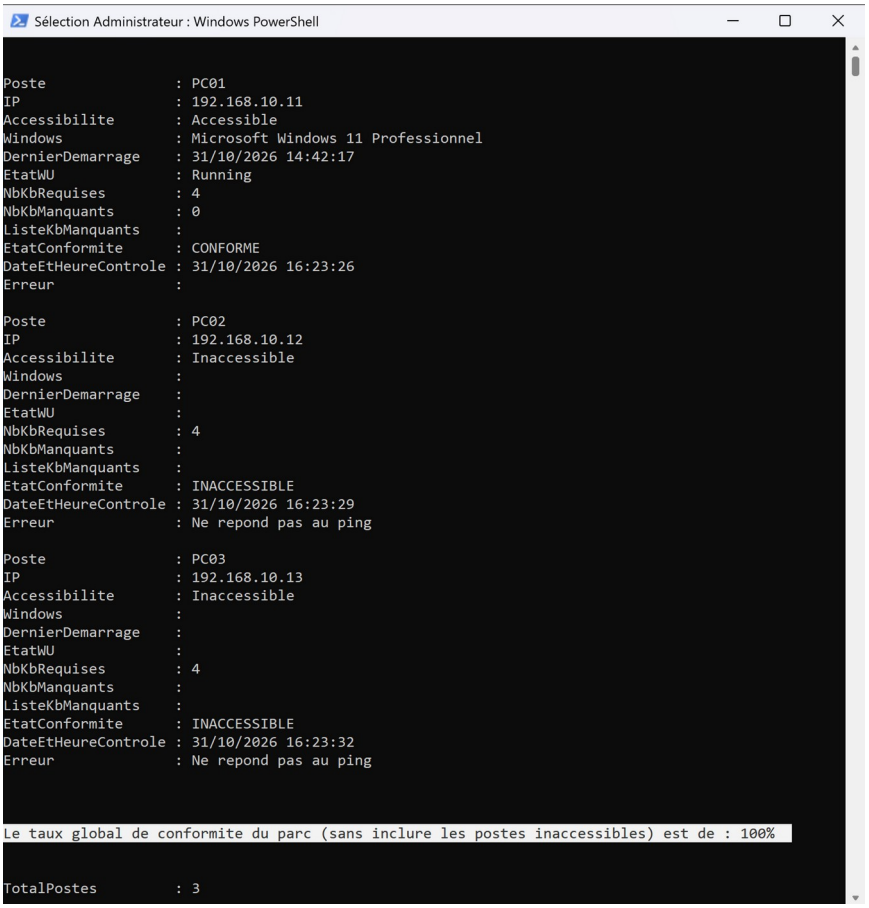


Figure 14 : PatchManager.ps1, cas CONFORME (détail par poste)

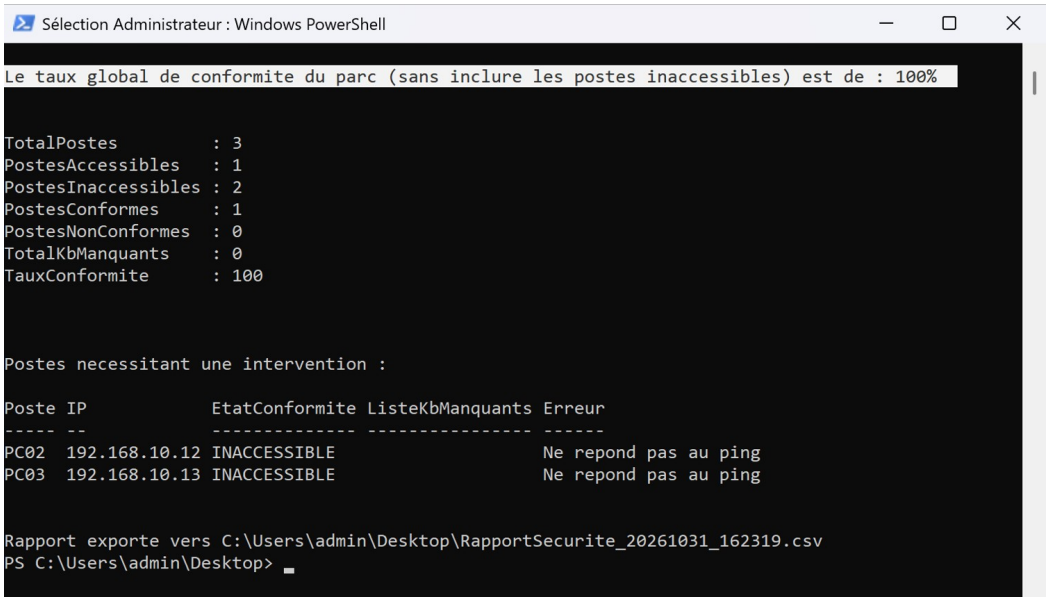


Figure 15 : PatchManager.ps1, cas CONFORME (synthèse)

5.3 Partie 7 : notification

Nous avons lancé le serveur de test dans une fenêtre, puis Send-Notification.ps1 dans une autre. Le mail arrive avec l'objet, les chiffres et le détail par poste demandés par le sujet.

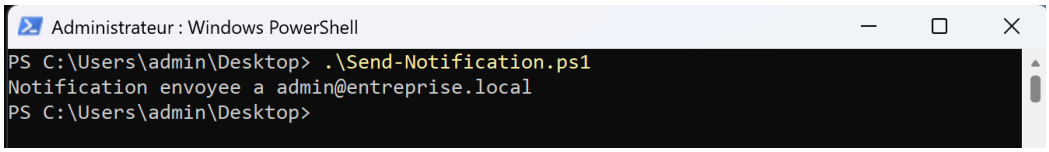


Figure 16 : Send-Notification.ps1 : envoi de la notification

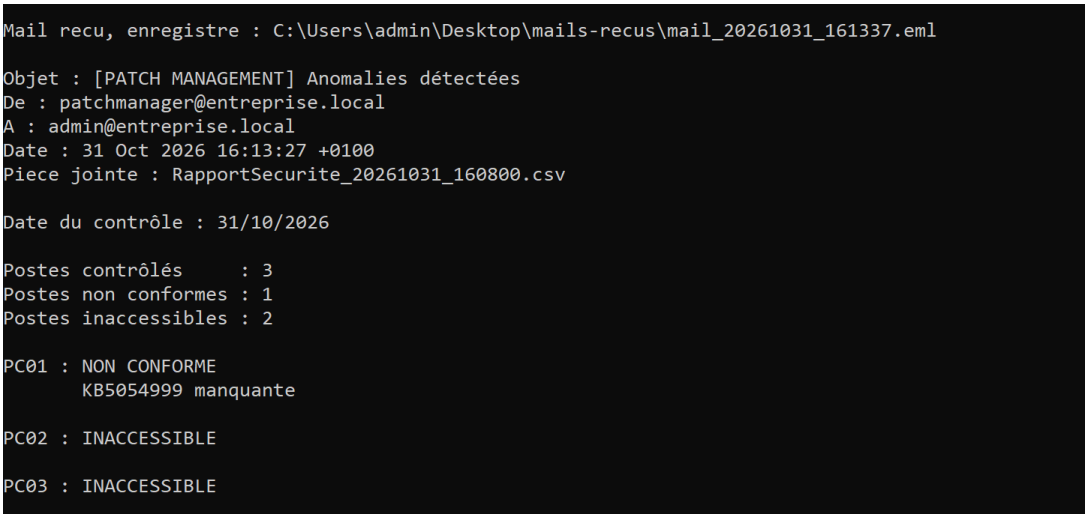


Figure 17 : Serveur SMTP de test : le mail reçu s'affiche directement lisible

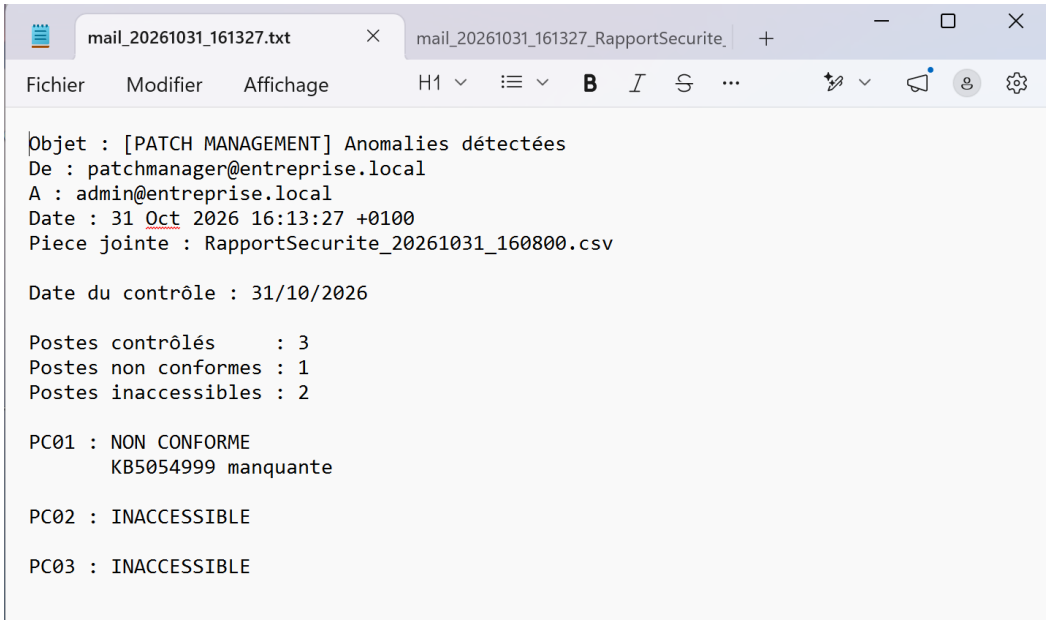


Figure 18 : Version texte du mail enregistrée par le serveur de test

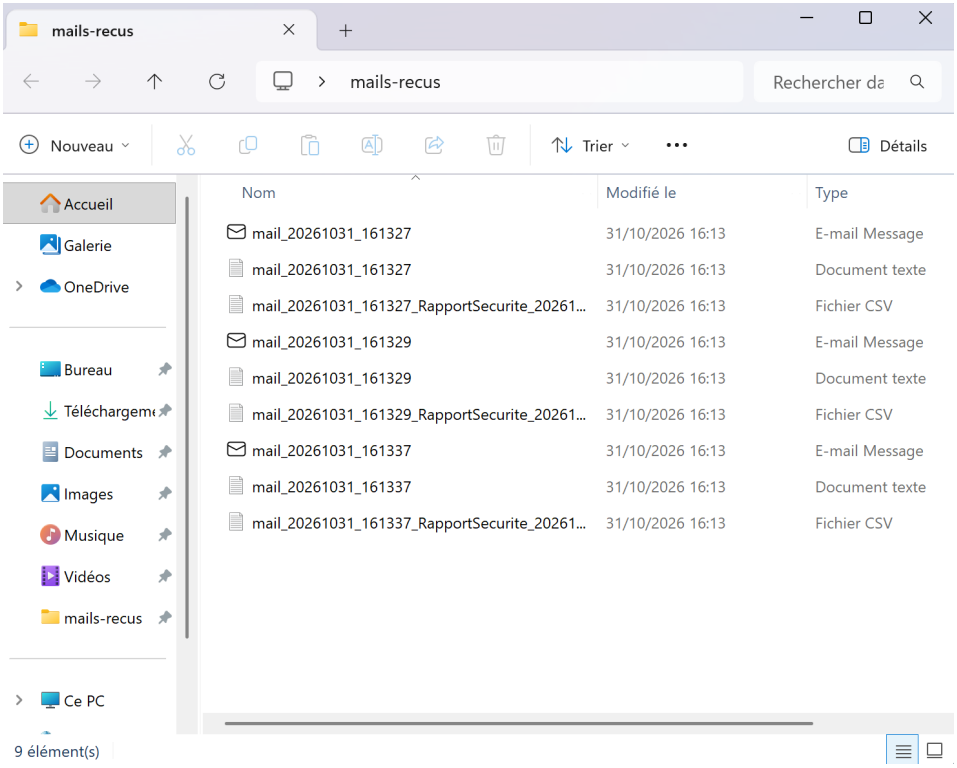


Figure 19 : Dossier mails-recus : le mail brut (.eml), sa version lisible (.txt) et la pièce jointe (.csv)

Quand le serveur de test est arrêté, le script ne plante pas : le message d'erreur est noté dans le journal et la notification est marquée comme non envoyée. La capture suivante montre les deux cas dans PatchManager.log : l'erreur « Impossible de se connecter au serveur distant » quand le serveur est arrêté, puis « Notification envoyée à admin@entreprise.local » quand il tourne.

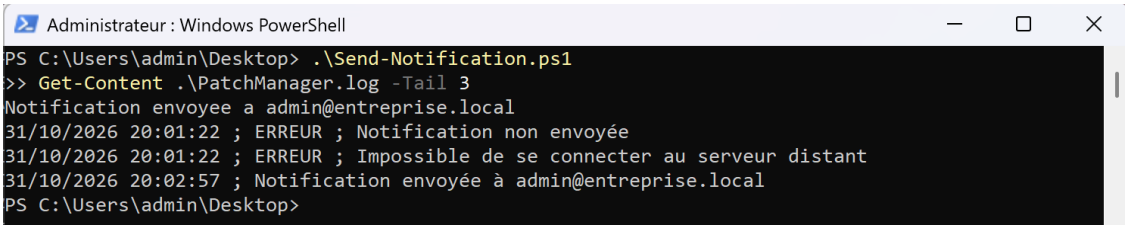


Figure 20 : Journal PatchManager.log : notification en échec puis notification envoyée

6. Sécurisation de la solution

Cette partie analyse les risques de l'administration à distance et propose des mesures. Nous avons fait les vérifications sur nos deux machines Voici l'essentiel, les réponses complètes aux huit questions sont en annexe D.

6.1 Risques de WinRM

PowerShell Remoting est un outil légitime, ce qui le rend dangereux entre de mauvaises mains. Les risques principaux sont le vol ou la devinette du mot de passe d'un compte administrateur, un trafic moins protégé en NTLM (sans domaine) qu'en Kerberos, une liste TrustedHosts trop large (le caractère * accepte n'importe quelle machine), le rebond d'un poste compromis vers les autres, le peu de traces laissées par défaut, et un service qui écoute en permanence sur le réseau.

6.2 Qui peut ouvrir une session distante

Sur PC01, trois comptes sont administrateurs locaux : admin, Administrateur et PC001. Le compte Administrateur intégré est désactivé, mais les deux autres sont actifs et peuvent ouvrir une session distante. La configuration de session Microsoft.PowerShell autorise les administrateurs, le groupe « Utilisateurs de gestion à distance » et le groupe INTERACTIF.

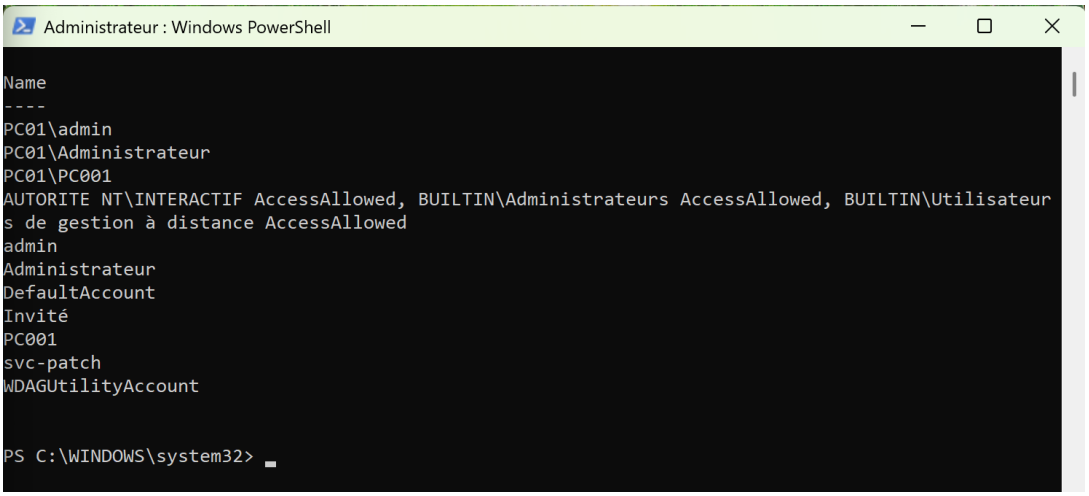


Figure 21 : PC01 : administrateurs locaux, permissions de la session distante et comptes locaux

6.3 Limiter les privilèges du compte

Notre script utilise le compte admin, administrateur complet, alors qu'il ne fait que lire. Pour appliquer le moindre privilège, nous avons créé sur PC01 un compte dédié, svc-patch, qui n'est pas administrateur et fait seulement partie du groupe « Utilisateurs de gestion à distance ». Nous l'avons testé depuis ADMIN01. Par défaut il peut ouvrir une session et lire le service Windows Update, mais Get-HotFix et Get-CimInstance sont refusés, parce que l'espace de noms WMI root\cimv2 est réservé aux administrateurs.

Nous avons alors donné à svc-patch, sur root\cimv2, uniquement les droits « Activer le compte » et « Appel à distance autorisé ». Get-CimInstance fonctionne ensuite, mais Get-HotFix renvoie 0 correctif pour svc-patch, sans erreur, alors que le compte admin en voit 5 sur le même poste. La liste des correctifs n'est lisible que par un administrateur.

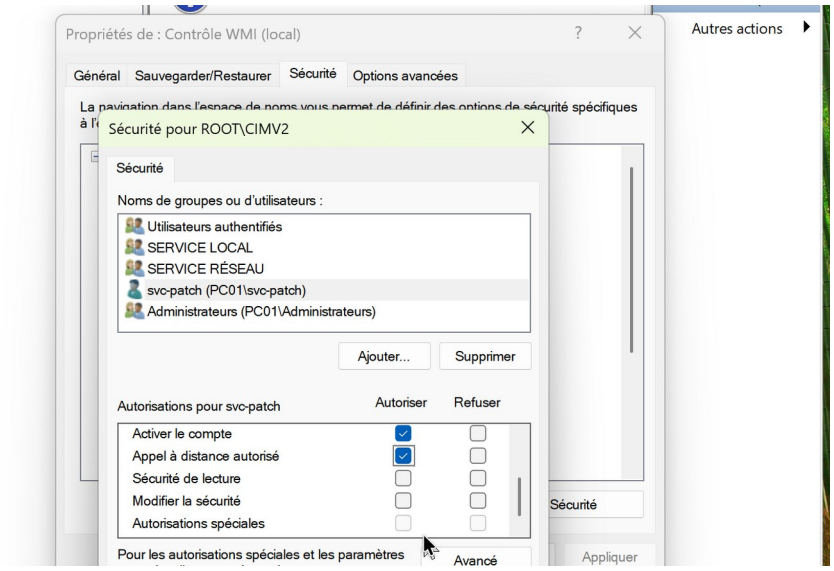


Figure 22 : Droits WMI accordés à svc-patch sur ROOT\CIMV2

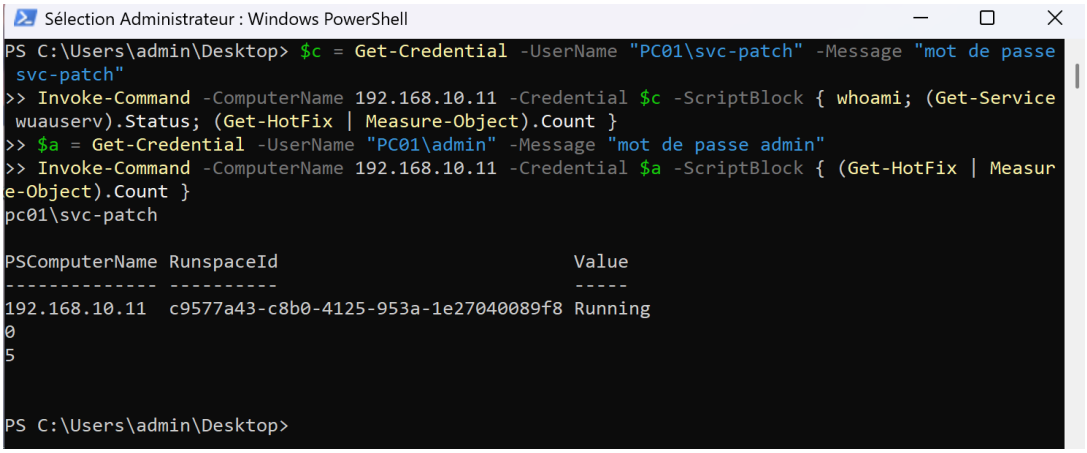


Figure 23 : Tests de svc-patch depuis ADMIN01 : session et Windows Update fonctionnent, mais 0 KB visible contre 5 pour admin

Ce résultat nous a appris qu'un compte trop limité ne fait pas forcément d'erreur : il peut donner une mauvaise réponse (0 KB), ce qui ferait classer un poste NON CONFORME à tort. Nous avons donc gardé le compte admin pour la démonstration et présentons svc-patch comme la cible à atteindre avec JEA (point 6.7).

6.4 Accès aux fichiers produits

Les droits (icacls) du journal et d'un rapport CSV sur ADMIN01 ne montrent que trois entrées en contrôle total : SYSTEM, le groupe Administrateurs et le compte admin, avec des droits hérités du dossier. Le groupe Utilisateurs n'y figure pas, donc un utilisateur standard ne peut pas lire ces fichiers. La limite est que le compte admin peut les modifier ou les supprimer.

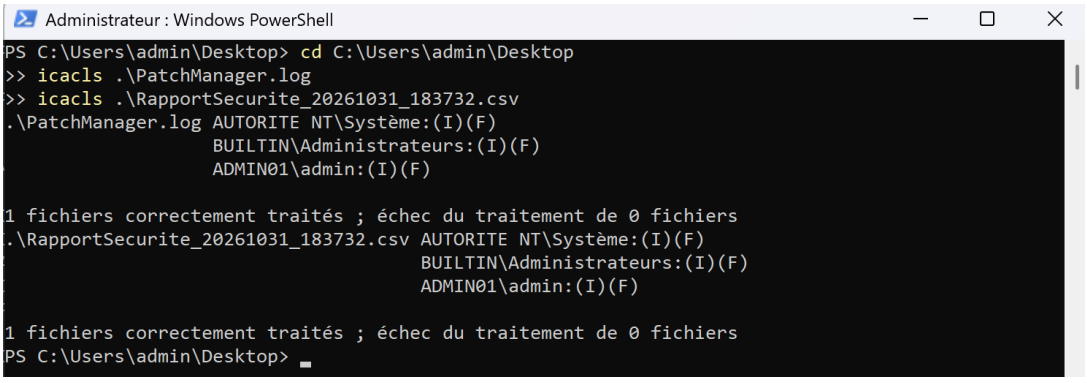


Figure 24 : Droits icacls sur PatchManager.log et sur un rapport CSV (ADMIN01)

6.5 Règles du pare-feu

Sur PC01, la règle WINRM-HTTP-In-TCP (profil Public, depuis le sous-réseau local) est désactivée, et la règle WINRM-HTTP-In-TCP-NoScope (profils Domaine et Privé, port 5985) est activée avec « Depuis : Any », donc ouverte à toutes les adresses. Nous l'avons restreinte à l'adresse d'ADMIN01 avec Set-NetFirewallRule. Après la restriction, le test WinRM et PatchManager.ps1 fonctionnent toujours depuis ADMIN01.



Figure 25 : Règle de pare-feu WinRM de PC01 restreinte à 192.168.10.10

6.6 Authentification et Kerberos

Sur PC01, Basic est désactivé, Kerberos et Negotiate sont actifs et le trafic non chiffré est interdit. Sur ADMIN01 (le client), Basic, Digest, Kerberos, Negotiate et Certificate sont autorisés, CredSSP est désactivé, et TrustedHosts contient seulement l'adresse de PC01. Nos deux machines sont en groupe de travail : sans contrôleur de domaine, Kerberos n'est pas utilisable et Negotiate se rabat sur NTLM, d'où la nécessité de TrustedHosts.

Kerberos est préférable dès que les machines sont dans un domaine Active Directory : il authentifie les deux côtés (le client vérifie qu'il parle au vrai serveur), n'envoie pas le mot de passe sur le réseau (il utilise des tickets temporaires), résiste mieux aux attaques par relais et évite de gérer TrustedHosts. À défaut de domaine, WinRM en HTTPS avec un certificat serait la meilleure option.

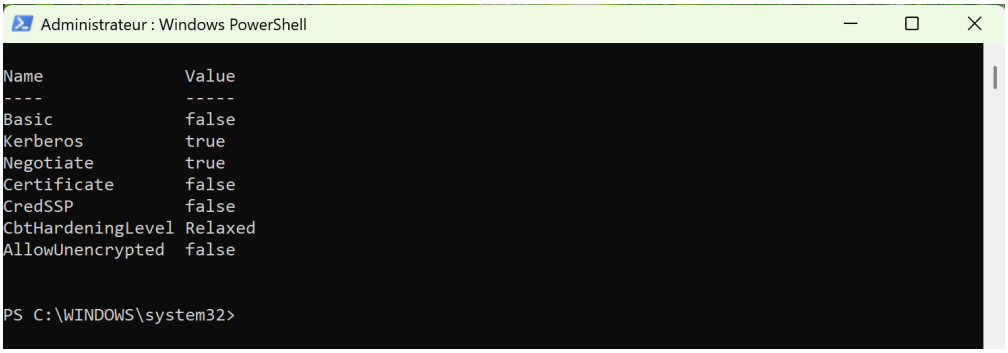


Figure 26 : Authentification WinRM côté serveur (PC01)

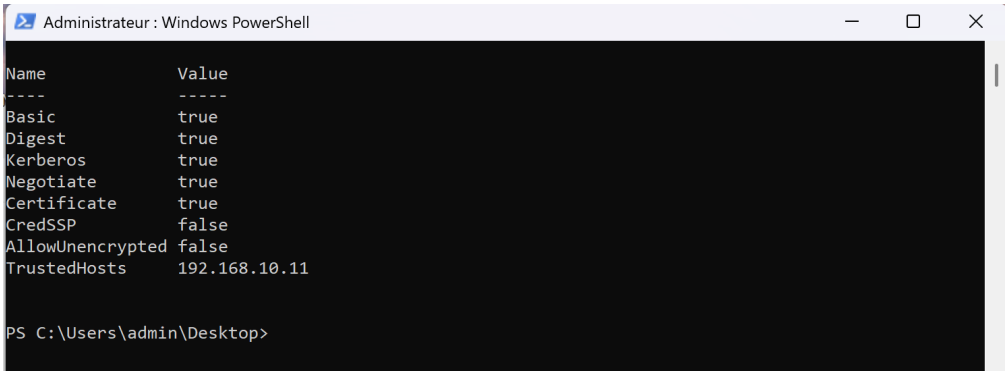


Figure 27 : Authentification et TrustedHosts côté client (ADMIN01)

6.7 Just Enough Administration (JEA)

JEA permet de donner à un utilisateur le droit de lancer seulement certaines commandes à distance, sans le faire administrateur. On définit un fichier de capacités de rôle (.psrc) qui liste les commandes et paramètres autorisés, et un fichier de configuration de session (.pssc) qui associe un groupe d'utilisateurs à ce rôle. La session s'exécute avec un compte virtuel élevé créé pour

l'occasion, donc le compte qui se connecte n'a aucun droit d'administration. Les commandes peuvent être journalisées.

Dans notre projet, JEA règlerait le problème vu avec svc-patch : un point de connexion limité pourrait autoriser seulement Get-Service (sur wuauserv), Get-HotFix et Get-CimInstance sur Win32_OperatingSystem, et le script s'y connecterait avec Invoke-Command -ConfigurationName. Si le compte était volé, l'attaquant ne pourrait rien faire d'autre. Nous avons étudié le principe et l'avons appliqué à notre cas, mais nous ne l'avons pas installé dans notre laboratoire.

6.8 Conservation des rapports et des journaux

Les rapports et le journal contiennent la liste des postes et des correctifs manquants, c'est-à-dire une carte des machines vulnérables. Nous proposons de les ranger dans un dossier dédié dont on coupe l'héritage des droits, avec l'écriture réservée au compte du script et la lecture aux administrateurs et auditeurs, de calculer une empreinte (Get-FileHash) de chaque rapport pour détecter une modification, et d'activer l'audit des accès aux fichiers. Contre la suppression, il faut envoyer une copie vers un emplacement distant en écriture seule ou vers un serveur de journaux, et garder les journaux environ 12 mois et les rapports détaillés 90 jours avant de ne conserver que les synthèses. Contre l'exposition, il faut chiffrer le disque, éviter les canaux non chiffrés pour envoyer les rapports (TLS pour le SMTP) et ne jamais écrire d'identifiants dans les journaux.

7. Difficultés rencontrées

- Première image Windows téléchargée en x64, incompatible avec l'architecture ARM de notre Mac : il a fallu reprendre l'image ARM64 officielle.
- Comparaison des listes de KB avec -notcontains : le tableau doit être placé à gauche de l'opérateur, ce qui nous a fait perdre du temps.
- Un deuxième Invoke-Command dans le bloc distant ne fonctionnait pas, car les variables locales n'y existent pas. Nous récupérons maintenant la liste des KB une fois et comparons en local.
- Une faute de frappe sur une variable ($htmlPostes) laissait vide la section « Détail par poste » du rapport HTML, sans aucune erreur PowerShell. Nous avons appris à vérifier visuellement les sorties.
- Le profil réseau de PC01 repasse en Public après un redémarrage, ce qui bloque WinRM tant qu'on ne le remet pas en Privé avec Set-NetConnectionProfile.
- Sans Internet sur les machines virtuelles, impossible d'installer un serveur SMTP de test : nous avons écrit le nôtre en PowerShell.
- Un caractère accentué du journal (le « à » de « envoyée à ») avait été écrit avec le code du « é » : la ligne affichait « envoyée é ». Corrigé avec le bon code ([char]0xE0).
- Avec le compte svc-patch, Get-HotFix renvoyait 0 correctif sans erreur : un compte trop limité peut produire une réponse fausse sans prévenir.

8. Limites et améliorations possibles

- Le script de notification ne se souvient pas de ce qu'il a déjà signalé : si une anomalie dure, l'administrateur reçoit le même mail à chaque exécution.

- Send-MailMessage est marqué comme obsolète par Microsoft. En production, il vaudrait mieux passer par une API de messagerie (OAuth) ou un canal Teams ou Slack.
- Le compte admin est encore utilisé par le script. Une solution complète utiliserait JEA avec un compte dédié, que nous avons étudié sans l'installer.
- WinRM reste en HTTP. Dans un vrai parc, il faudrait un domaine avec Kerberos, ou HTTPS avec certificat.
- Les journaux peuvent être modifiés par un administrateur local. Ils devraient être envoyés vers un serveur de journaux ou un SIEM.
- Le parc de test ne compte qu'un poste réel. Nous n'avons pas testé la durée d'exécution sur un grand nombre de machines, où il faudrait lancer les connexions en parallèle.

9. Conclusion

Notre solution contrôle à distance l'état des correctifs d'un parc, calcule la conformité, produit un rapport CSV et HTML, prévient l'administrateur uniquement en cas d'anomalie et garde une trace de chaque action dans un journal. Elle a été testée sur nos deux machines virtuelles, avec des cas conformes, non conformes et injoignables.

Ce projet nous a surtout montré que l'administration à distance doit être pensée avec la sécurité dès le départ : limiter les comptes, restreindre le pare-feu, choisir le bon mode d'authentification et protéger les journaux. Avec plus de temps, nous mettrions en place JEA, un domaine avec Kerberos, HTTPS pour WinRM et un envoi des journaux vers un serveur central.

Annexes : réponses aux questions du sujet

Cette annexe regroupe les réponses aux questions posées à la fin des parties 1, 4, 7 et 8 du sujet.

Annexe A : Partie 1, préparation de l'environnement

Quel est le rôle de WinRM dans PowerShell Remoting ?

WinRM (Windows Remote Management) est le protocole de communication de PowerShell Remoting. Il permet d'exécuter des commandes et des scripts PowerShell sur un ordinateur distant sans avoir besoin d'un accès physique. C'est l'équivalent de SSH sur Windows. Quand on utilise Invoke-Command ou Enter-PSSession, c'est WinRM qui transporte la requête jusqu'à la machine distante, qui l'exécute puis renvoie le résultat.

Quelle commande permet d'activer PowerShell Remoting ?

La commande à exécuter sur la machine cible, en administrateur, est Enable-PSRemoting. Elle démarre le service WinRM, crée un écouteur, active la règle de pare-feu correspondante et enregistre les configurations de session par défaut.

Quels ports réseau sont utilisés par WinRM ?

WinRM utilise le port 5985 pour le trafic HTTP, utilisé par défaut (la charge utile reste toutefois chiffrée lors de l'authentification Windows), et le port 5986 pour le trafic HTTPS, chiffré avec SSL/TLS et qui nécessite un certificat.

Pourquoi le pare-feu Windows peut-il empêcher une connexion PowerShell distante ?

Par défaut, la politique du pare-feu Windows bloque les flux entrants non sollicités, y compris sur les ports TCP 5985 et 5986. De plus, si l'interface réseau est assignée au profil réseau Public, Windows refuse d'ouvrir ces exceptions pour des raisons de sécurité. La connexion échoue tant que la règle de trafic entrant pour Windows Remote Management n'est pas active sur le profil réseau adéquat (Privé ou Domaine). C'est ce que nous avons constaté sur PC01, dont le profil repasse en Public après un redémarrage.

Quelle différence existe-t-il entre tester la connectivité avec Test-Connection et tester WinRM avec Test-WSMan ?

Test-Connection teste la connexion entre la machine d'administration et la machine cible avec une requête ICMP (ping). Il dit seulement que la machine répond sur le réseau. Test-WSMan envoie une demande d'identification qui détermine si le service WinRM s'exécute sur l'ordinateur local ou distant. Une machine peut donc répondre au ping tout en refusant WinRM.

Pourquoi est-il préférable de vérifier l'accessibilité d'une machine avant de tenter une commande distante ?

Cela permet de savoir si WinRM est bien activé, et de gagner du temps si la machine est injoignable : une commande distante sur un poste éteint attend un long délai avant d'échouer. C'est pour cela que nos scripts testent d'abord la machine avec un ping.

Quels problèmes de sécurité peuvent apparaître si PowerShell Remoting est activé sans restriction ?

L'activation sans restriction de PowerShell Remoting expose à plusieurs risques :

- **Mouvement latéral et élévation de privilèges :** si les accès ne sont pas restreints aux comptes d'administration légitimes, un attaquant disposant d'identifiants compromis peut rebondir d'une machine à une autre et exécuter du code à distance.
- **Ports ouverts sans filtrage :** laisser les ports 5985 et 5986 ouverts sans filtrage IP permet à n'importe quel équipement du sous-réseau de tenter de l'énumération ou des attaques par force brute sur les comptes locaux.

Annexe B : Partie 4, contrôle de conformité

Pourquoi la politique de conformité doit-elle être régulièrement mise à jour ?

Une politique de conformité doit être mise à jour régulièrement parce que Microsoft publie de nouveaux correctifs de sécurité tous les mois (le Patch Tuesday), à mesure que des vulnérabilités sont découvertes. Si la liste de required-patches.txt reste figée, le script ne vérifie que les anciens correctifs et ne voit pas les nouveaux. Un poste peut alors être affiché CONFORME alors qu'il lui manque des mises à jour récentes, ce qui donne une fausse impression de sécurité sur tout le parc.

Il y a aussi le cas des mises à jour cumulatives : quand une nouvelle KB remplace une ancienne, l'ancienne peut disparaître de la liste des correctifs installés, et un poste pourtant à jour risque d'être marqué NON CONFORME à tort. C'est donc à l'administrateur sécurité de revoir la liste à intervalles réguliers, en s'appuyant sur les bulletins de Microsoft (Security Update Guide) et sur la criticité des failles, pour que le contrôle reflète le vrai niveau de protection. Mettre à jour les systèmes ne protège pas de tout (le social engineering reste possible), mais cela réduit fortement la surface d'attaque.

Annexe C : Partie 7, notification

Question 2 : Quels sont les inconvénients de cette solution ?

La solution marche, mais elle a plusieurs limites que nous avons pu constater en la testant.

Elle dépend entièrement du serveur SMTP. Si le serveur est éteint, injoignable ou bloqué par un pare-feu, le mail ne part pas. Nous l'avons vu pendant nos tests : sans notre serveur de test lancé, le script a écrit "Notification non envoyée" dans le journal avec l'erreur "Impossible de se connecter au serveur distant". Dans ce cas l'administrateur n'est pas prévenu, alors que c'est justement le but du script. Le journal garde une trace de l'échec, mais personne ne le lit en temps réel.

La gestion du mot de passe reste fragile. Le fichier smtp-cred.xml créé avec Export-Clixml est chiffré avec DPAPI : il ne peut être relu que par le même utilisateur sur la même machine. C'est mieux qu'un mot de passe en clair dans le script, mais ça veut dire qu'il faut recréer le fichier si on change de compte ou de machine, et que n'importe quel processus lancé avec ce compte peut le déchiffrer.

Le mail n'est pas fiable à 100 %. Un mail peut finir dans les spams, arriver avec du retard, ou ne jamais être lu. Il n'y a pas d'accusé de réception : le script sait que le serveur a accepté le message, pas que l'administrateur l'a vu. Et sans chiffrement (TLS), le contenu circule en clair, alors qu'il donne la liste des postes vulnérables et des KB manquantes. Pour un attaquant qui l'intercepte, c'est une liste de cibles.

Il y a un risque de trop de notifications. Si le script tourne toutes les heures et qu'un poste reste inaccessible pendant une semaine, l'administrateur reçoit le même mail 168 fois. Il finit par ne plus les lire. Notre script ne mémorise pas ce qui a déjà été signalé.

Le script n'est pas très portable. Send-MailMessage est marqué comme obsolète par Microsoft : il ne gère pas bien les protocoles modernes d'authentification. Et l'adresse du serveur, l'expéditeur et le destinataire sont écrits directement dans le script, ce qui n'est pas pratique à maintenir.

Question 3 : Quelles alternatives modernes en environnement réel ?

Dans une vraie entreprise, on n'enverrait pas un mail depuis un script comme ça. On utiliserait plutôt :

- **Une messagerie d'équipe avec webhook** (Microsoft Teams, Slack) : le script envoie

un message à un canal avec une requête HTTPS, et toute l'équipe d'administration le voit tout de suite, sur son téléphone aussi.

- **Une API de messagerie moderne** (Microsoft Graph, SendGrid) : l'envoi passe par un

jeton OAuth2 au lieu d'un mot de passe, et on peut le révoquer ou limiter ses droits.

- **Un outil de supervision ou un SIEM** (Zabbix, Nagios, Splunk, Microsoft Sentinel) :

les résultats sont envoyés comme des événements, avec des seuils, des alertes qui s'escaladent et un historique. Ça évite aussi le problème des mails répétés.

- **Un coffre-fort de secrets** (Azure Key Vault, HashiCorp Vault, module

SecretManagement de PowerShell) pour stocker les identifiants, au lieu d'un fichier sur le disque.

- **Un outil de gestion de tickets** (GLPI, ServiceNow, Jira) : une anomalie crée

automatiquement un ticket avec un responsable et un suivi jusqu'à la résolution.

- **Une solution de gestion des correctifs dédiée** (WSUS, Intune, SCCM/Configuration

Manager) qui fait déjà le suivi de conformité et les rapports, sans que l'on code tout nous-mêmes.

L'idée générale est de passer d'un mail "envoyé et oublié" à un système qui authentifie proprement, qui trace, et qui s'assure que quelqu'un traite l'anomalie.

Annexe D : Partie 8, sécurisation de la solution

Environnement de test : ADMIN01 (192.168.10.10), le poste d'administration, et PC01 (192.168.10.11), le poste administré. Les deux machines sont en groupe de travail, sans domaine.

1. Risques de PowerShell Remoting et de WinRM

PowerShell Remoting permet de lancer des commandes sur une autre machine. C'est exactement ce qui le rend dangereux : si quelqu'un en prend le contrôle, il peut s'en servir pour attaquer tout le parc. Voici les risques principaux que nous avons retenus.

Le premier est le vol ou la devinette du mot de passe. Une session WinRM s'ouvre avec un compte administrateur, donc si son mot de passe est faible, un attaquant peut se connecter à distance et faire ce qu'il veut avec les droits admin. Dans notre labo, le mot de passe de PC01 est très simple, ce qui n'est acceptable que pour un test.

Le deuxième est le trafic qui n'est pas chiffré par défaut. WinRM écoute en HTTP sur le port 5985. Dans un domaine, Kerberos protège l'authentification, mais en groupe de travail, comme chez nous, on est en NTLM et la protection est plus faible. Pour tout chiffrer, il faudrait passer en HTTPS (port 5986) avec un certificat.

Le troisième est la liste TrustedHosts. Sans domaine, il faut déclarer les machines de confiance. Si on met *, on accepte n'importe quelle machine, et une fausse machine pourrait se faire passer pour PC01 et récupérer nos identifiants. Il vaut mieux lister les adresses exactes.

Le quatrième est le déplacement latéral. Si un poste ou un compte admin est compromis, WinRM permet de rebondir vers les autres machines avec un outil normal de Windows, sans rien installer, ce qui est difficile à repérer.

Le cinquième est le manque de traces. Par défaut, les commandes lancées à distance laissent peu d'informations. Sans journalisation (Script Block Logging, transcription), on ne sait pas qui a exécuté quoi.

Le dernier est la surface d'attaque : le service écoute en permanence sur le réseau, et si le pare-feu l'ouvre à tout le monde, n'importe quelle machine peut essayer de s'y connecter.

2. Qui peut ouvrir une session distante

Pour le vérifier, nous avons interrogé PC01 avec Get-LocalGroupMember, Get-LocalUser et Get-PSSessionConfiguration.

Le groupe Administrateurs de PC01 contient trois comptes : admin, Administrateur et PC001. Get-LocalUser montre que le compte Administrateur, le compte intégré de Windows, est désactivé, ce qui est une bonne chose. Il reste donc deux comptes administrateurs actifs, admin et PC001. Le groupe « Utilisateurs de gestion à distance » était vide au départ.

La configuration de session Microsoft.PowerShell autorise les Administrateurs, les Utilisateurs de gestion à distance et le groupe INTERACTIF. Les sessions microsoft.powershell.workflow et microsoft.powershell32 ont des droits similaires.

Notre conclusion est que l'accès distant n'est pas limité au strict nécessaire. Notre script utilise le compte admin, qui est administrateur complet, alors qu'il ne fait que lire des informations. Si ce compte est volé, l'attaquant a la main totale sur le poste. De plus, PC001 est un second compte administrateur actif qui ne sert à rien dans le projet mais qui peut quand même ouvrir une session distante.

Pour améliorer ça, on peut utiliser un compte dédié au patch management (point 3), retirer les droits d'administrateur aux comptes qui n'en ont pas besoin, et restreindre la session distante avec JEA (point 7).

3. Limiter les privilèges du compte de patch management

Pour appliquer le moindre privilège, nous avons créé sur PC01 un compte local dédié, svc-patch. Il n'est pas administrateur, il est seulement dans le groupe « Utilisateurs de gestion à distance », qui permet d'ouvrir une session distante. Nous l'avons testé depuis ADMIN01 avec Invoke-Command, en le comparant au compte admin.

Premier test, avec les droits par défaut : l'ouverture de session fonctionne (whoami renvoie pc01\svc-patch) et la lecture du service Windows Update aussi (Get-Service wuauserv). En revanche,

Get-HotFix et Get-CimInstance Win32_OperatingSystem donnent « Accès refusé », parce que l'espace de noms WMI root\cimv2 est réservé aux administrateurs.

Nous avons donc ajouté à svc-patch, sur root\cimv2 (dans wmimgmt.msc, onglet Sécurité), uniquement les droits « Activer le compte » et « Appel à distance autorisé », appliqués à l'espace de noms et à ses sous-espaces. Après ça, Get-CimInstance Win32_OperatingSystem fonctionne avec svc-patch. Mais Get-HotFix renvoie 0 correctif, sans aucune erreur, alors que le compte admin en renvoie 5 sur le même poste.

Ce qu'on en retient, c'est que la liste des correctifs installés n'est lisible que par un administrateur. Surtout, un compte trop limité ne fait pas forcément d'erreur : il peut donner une mauvaise réponse (0 KB), et dans notre rapport un poste serait classé NON CONFORME à tort. Nous n'avons donc pas basculé PatchManager.ps1 sur svc-patch, et nous avons gardé le compte admin pour la démonstration.

La vraie solution pour respecter le moindre privilège est JEA (point 7) : un point de connexion limité qui n'expose que les commandes nécessaires, en lecture seule, et qui s'exécute côté serveur avec un compte virtuel. Le compte qui se connecte reste alors sans droits d'administration.

4. Accès aux fichiers produits

Nous avons vérifié avec icacls les droits sur PatchManager.log, sur les rapports CSV (RapportSecurite_*.csv) et sur le dossier qui les contient, sur ADMIN01. Seuls trois comptes y ont accès, tous en contrôle total : SYSTEM, le groupe Administrateurs et le compte admin. Les droits sont hérités du dossier. Le groupe Utilisateurs n'apparaît pas, donc un utilisateur standard de la machine ne peut pas lire ces fichiers.

La limite est que le compte admin peut modifier ou supprimer le journal et les rapports. Dans une vraie exploitation, il faudrait ranger les fichiers dans un dossier dédié (reports\), couper l'héritage avec icacls /inheritance:r, donner la lecture seule aux auditeurs et l'écriture au compte du script seulement (voir point 8).

5. Règles du pare-feu Windows pour WinRM

Nous avons listé les règles de pare-feu WinRM sur PC01 avec Get-NetFirewallRule, Get-NetFirewallAddressFilter et Get-NetFirewallPortFilter. La règle WINRM-HTTP-In-TCP (profil Public, port 5985, depuis le sous-réseau local) est désactivée. La règle WINRM-HTTP-In-TCP-NoScope (profils Domaine et Privé, port 5985, depuis n'importe quelle adresse) est activée.

Le réseau de PC01 est en profil Privé, donc c'est la règle NoScope qui s'applique. Elle acceptait les connexions WinRM depuis n'importe quelle adresse IP, alors que seul ADMIN01 en a besoin. C'était trop permissif : n'importe quelle machine du réseau pouvait essayer de se connecter à PC01.

Nous avons donc restreint cette règle à l'adresse 192.168.10.10 (ADMIN01) avec Set-NetFirewallRule -RemoteAddress. Après ce changement, Test-WSMan et Invoke-Command depuis ADMIN01 fonctionnent toujours, PatchManager.ps1 produit le même rapport, et les autres adresses du réseau n'ont plus accès au port 5985 de PC01.

6. Mode d'authentification et Kerberos

Nous avons relu la configuration WinRM des deux machines (WSMan:\localhost\Service\Auth sur PC01 et WSMan:\localhost\Client\Auth sur ADMIN01).

Sur PC01, côté serveur, Basic est désactivé, Kerberos et Negotiate sont activés, Certificate et CredSSP sont désactivés, et AllowUnencrypted est à false. Sur ADMIN01, côté client, Negotiate, Kerberos, Digest, Certificate et Basic sont activés, CredSSP est désactivé et AllowUnencrypted est à false. TrustedHosts contient seulement 192.168.10.11.

Nos deux machines ne sont pas dans un domaine. C'est pour ça que Windows refusait la connexion tant que l'adresse de PC01 n'était pas dans TrustedHosts. Sans domaine, il n'y a pas de contrôleur de domaine pour délivrer les tickets, donc Kerberos ne peut pas servir, et Negotiate se replie sur NTLM.

Côté points positifs, Basic (mot de passe presque en clair) est désactivé sur le serveur et le trafic non chiffré est interdit. Avec Negotiate, le contenu des messages WinRM est chiffré même en HTTP. Côté points faibles, NTLM est moins solide que Kerberos (attaques par relais, mot de passe attaquable hors ligne), et le client n'authentifie pas le serveur, d'où TrustedHosts. Il serait aussi préférable de désactiver Basic côté client.

Kerberos est préférable dès que les machines sont dans un domaine Active Directory. Il authentifie les deux côtés (le client vérifie qu'il parle au vrai serveur, ce que NTLM ne fait pas), n'envoie jamais le mot de passe sur le réseau parce qu'il utilise des tickets temporaires, résiste mieux aux attaques par relais et par rejeu, permet l'authentification unique et évite de gérer TrustedHosts. En production, nous utiliserions donc un domaine avec Kerberos, ou à défaut WinRM en HTTPS avec un certificat.

7. Just Enough Administration (JEA)

JEA est une fonction de PowerShell qui permet de donner à un utilisateur le droit de lancer seulement certaines commandes à distance, sans le faire administrateur. C'est le moindre privilège appliqué aux commandes : on ne donne pas une session d'administration complète, mais un point d'entrée limité.

Pour le mettre en place, on crée un fichier de capacités de rôle (.psrc, avec New-PSRoleCapabilityFile) qui liste les cmdlets, fonctions et paramètres autorisés. On crée ensuite un fichier de configuration de session (.pssc, avec New-PSSessionConfigurationFile) qui associe un groupe d'utilisateurs à ce rôle et qui met le mode du langage en restreint. On l'enregistre sur le poste avec Register-PSSessionConfiguration. La session s'exécute ensuite avec un compte virtuel élevé, créé pour l'occasion : le compte qui se connecte n'a pas de droits d'administrateur, mais la session peut quand même lire ce qui est protégé. Toutes les commandes peuvent aussi être journalisées.

Dans notre projet, JEA résoudrait le problème du point 3. Le compte svc-patch ne pourrait pas lire la liste des KB tout seul, mais un point de connexion « PatchReader » pourrait autoriser uniquement Get-Service (limité à wuauserv), Get-HotFix et Get-CimInstance sur Win32_OperatingSystem. Le script s'y connecterait avec Invoke-Command -ConfigurationName PatchReader. Si le compte était volé, l'attaquant ne pourrait lancer aucune autre commande, donc pas d'installation et pas de suppression de fichiers. Nous n'avons pas mis JEA en place

dans notre labo, par manque de temps : nous avons étudié le principe et nous l'avons appliqué à notre cas.

8. Stratégie de conservation des rapports et des journaux

Les rapports et le journal contiennent des informations sensibles. La liste des postes et des correctifs manquants est, pour un attaquant, une carte des machines vulnérables. Voici la stratégie que nous proposons.

Pour éviter la modification non autorisée, il faut ranger les fichiers dans un dossier dédié (reports\) et couper l'héritage des droits avec icacls /inheritance:r. Le compte du script ne devrait avoir que le droit d'ajouter des lignes, les administrateurs la lecture, et personne d'autre aucun droit. Le journal ne devrait être ouvert qu'en ajout, avec Add-Content. On peut aussi calculer l'empreinte de chaque rapport avec Get-FileHash et la stocker à part, pour détecter une modification, et activer l'audit des accès aux fichiers de Windows pour savoir qui a ouvert ou modifié quoi.

Pour éviter la suppression, il faut sauvegarder les rapports et les journaux sur un emplacement distant en écriture seule, ou sur un serveur de journaux (SIEM), pour qu'un attaquant qui prend ADMIN01 ne puisse pas effacer ses traces. Il faut aussi retirer le droit de suppression aux comptes courants. Pour la durée de conservation, nous proposons 12 mois pour les journaux, et 90 jours pour les rapports détaillés, avec ensuite une archive des synthèses seulement. La rotation est automatique : un nouveau fichier par jour, ce qui est déjà le cas pour les rapports grâce à la date dans le nom.

Pour éviter l'exposition d'informations sensibles, il faut limiter la lecture aux administrateurs et aux auditeurs, chiffrer le disque (BitLocker) et les archives, et ne pas envoyer les rapports par un canal non chiffré. Dans notre script de notification, le rapport part en pièce jointe, donc le serveur SMTP devrait utiliser TLS. Enfin, il ne faut jamais mettre de mots de passe dans les journaux : notre script ne journalise que des messages d'erreur, jamais d'identifiants.